

CYBERSECURITY DOMAIN PROJECT

Dissecting Packets: From HTTP to TLS

Understanding Wireshark — Packet Analysis Simplified

Subject	Cybersecurity / Network Analysis
Tool Used	Wireshark (Network Protocol Analyser)
Protocols Analysed	HTTP, DNS, TLS
Submission Deadline	10 May 2026, 11:59 PM (IST)

Submitted by: SHUVADEEP BASU

Date Submitted: 10.05.2026

1. Introduction

This report documents the hands-on packet analysis exercise conducted using Wireshark, a widely used open-source network protocol analyser. The objective of this project is to capture and inspect real network traffic, focusing on three key application-layer protocols: HTTP (Hypertext Transfer Protocol), DNS (Domain Name System), and TLS (Transport Layer Security).

Through this exercise, we aim to:

- Understand how data flows across the network at the packet level
- Apply Wireshark display filters to isolate specific protocol traffic
- Interpret packet headers, payloads, and handshake sequences
- Develop practical skills relevant to network troubleshooting and security monitoring

1.1 What is Wireshark?

Wireshark is a free and open-source packet analyser used for network troubleshooting, analysis, software development, and education. It captures packets from a network interface in real time and allows users to browse the traffic at a microscopic level. Wireshark uses display filters and colour-coding to make it easier to isolate and understand traffic of interest.

1.2 Lab Environment

Component	Details
Operating System	Windows 10 / Kali Linux (as applicable)
Wireshark Version	4.x (latest stable release)
Network Interface	Active Ethernet / Wi-Fi adapter
Capture Duration	Approximately 2–5 minutes per protocol
Target Sites	http://example.com (HTTP), https://example.com (HTTPS/TLS)

2. Methodology

The following step-by-step procedure was followed during this project:

1. Launch Wireshark and select the active network interface (e.g., eth0, Wi-Fi).
2. Start a live packet capture session.
3. Visit http://example.com to generate HTTP GET traffic.
4. Apply display filter http.request.method == "GET" to isolate HTTP requests.

5. Perform DNS lookups using nslookup or by visiting websites, then apply filter: dns.
6. Visit <https://example.com> and apply filter tls to observe the TLS handshake.
7. Document all observations, capture screenshots, and save the .pcap file.

SECTION A — HTTP PACKET ANALYSIS

3. HTTP GET Request Analysis

HTTP (HyperText Transfer Protocol) is a stateless, application-layer protocol used to transfer web content. When a user visits a non-HTTPS website, all communication is transmitted in plaintext, making it straightforward to capture and read in Wireshark.

3.1 Applying the HTTP Display Filter

To isolate HTTP GET requests from all captured traffic, the following Wireshark display filter was applied:

```
http.request.method == "GET"
```

3.3 Packet Details — HTTP GET

Expanding the Packet Details Pane for the captured HTTP GET request reveals the following information:

Field	Observed Value	Explanation
Source IP	192.168.x.x (your machine)	Client's private IP address
Destination IP	93.184.216.34	IP address of example.com's server
Source Port	Ephemeral (e.g., 54321)	Randomly assigned port by OS
Destination Port	80	Standard HTTP port
Request Method	GET	Client requests resource from server
Request URI	/	Root path of the website
HTTP Version	HTTP/1.1	Protocol version in use
Host Header	example.com	Domain the request is targeting
User-Agent	Mozilla/5.0 (varies)	Identifies the browser/client
Accept-Encoding	gzip, deflate	Supported compression methods

3.4 What the HTTP GET Packet Reveals

Because HTTP transmits data in plaintext, a packet capture reveals everything about the communication:

- The exact URL the user is visiting — including path and query parameters
- The browser type and version (from the User-Agent header)
- Accepted content formats and languages (from Accept headers)
- Session cookies (if present) — a significant security risk on plain HTTP
- Full response body from the server — HTML, JSON, or other content in cleartext



Security Insight: HTTP traffic is completely unencrypted. Any attacker on the same network (e.g., public Wi-Fi) can capture and read all HTTP traffic using tools like Wireshark. This is why HTTPS should always be used for sensitive communications.

SECTION B — DNS PACKET ANALYSIS

4. DNS Query and Response Analysis

DNS (Domain Name System) is the internet's phone book. It translates human-readable domain names (e.g., example.com) into IP addresses that computers use to communicate. DNS operates on UDP port 53 by default (and TCP port 53 for larger responses or zone transfers).

4.1 Applying the DNS Display Filter

```
dns
```

4.4 Packet Details — DNS Query

Field	Observed Value	Explanation
Source IP	192.168.x.x (your machine)	Initiating DNS query
Destination IP	8.8.8.8 (or ISP DNS)	DNS resolver (Google DNS or local)
Protocol	DNS over UDP	Uses UDP for speed
Source Port	Ephemeral	OS-assigned
Destination Port	53	Standard DNS port
Transaction ID	0xABCD (example)	Matches query to response
Query Type	A	Requests IPv4 address
Query Name	example.com	Domain being resolved
Flags	0x0100 (Standard query)	QR=0 (query), RD=1 (recursion desired)

4.5 Packet Details — DNS Response

Field	Observed Value	Explanation
Transaction ID	0xABCD (same)	Matches the original query
Response Code	NOERROR (0)	Resolution was successful
Answer Count	1 (or more)	Number of IP addresses returned

Resolved IP (A)	93.184.216.34	IPv4 address of example.com
TTL	3600 (example)	Seconds to cache this record
QR Flag	1 (Response)	This is a response packet
AA Flag	0	Not an authoritative answer
RA Flag	1	Recursion available at server

4.6 What the DNS Packets Reveal

- Every domain name a user visits — even on encrypted (HTTPS) sites — can be observed unless DNS-over-HTTPS (DoH) or DNS-over-TLS (DoT) is used
- The DNS resolver being used reveals information about the network configuration
- Repeated identical DNS queries can indicate automated/malicious software behaviour
- Unusually large DNS responses can indicate DNS amplification attacks
- Non-existent domain (NXDOMAIN) responses can reveal failed connections or malware callbacks

SECTION C — TLS HANDSHAKE ANALYSIS

5. TLS Handshake Analysis

TLS (Transport Layer Security) is the cryptographic protocol that secures HTTPS connections. Unlike HTTP, TLS encrypts all data exchanged after the handshake, making it impossible for an eavesdropper to read the actual content. However, the TLS handshake itself is observable and reveals important details about the negotiation process.

5.1 Applying the TLS Display Filter

```
tls
```

5.2 Overview — The TLS Handshake Process

The TLS handshake is a negotiation sequence that occurs before any application data is exchanged. It establishes encryption parameters and authenticates the server. The key steps are:

Step	Message	Direction	Purpose
1	Client Hello	Client → Server	Propose TLS version, cipher suites, and send random nonce
2	Server Hello	Server → Client	Select TLS version, cipher suite, and send server's nonce
3	Certificate	Server → Client	Send server's X.509 certificate for authentication
4	Server Hello Done	Server → Client	Signal end of server negotiation (TLS 1.2)
5	Client Key Exchange	Client → Server	Share key material (or public key)
6	Change Cipher Spec	Both	Signal switch to encrypted communication
7	Finished	Both	Verify handshake integrity; session begins

Figure 5.2 — Server Hello: selected cipher suite, server random, and session ID visible

5.5 Client Hello — Observed Details

Field	Observed Value	Explanation
-------	----------------	-------------

Handshake Type	1 (Client Hello)	Initiates the TLS handshake
TLS Version	TLS 1.2 / TLS 1.3	Highest supported version offered
Client Random	32 bytes (hex string)	Random value for key derivation
Session ID	0 or existing ID	0 = new session; value = resumption
Cipher Suites	Multiple listed	Algorithms the client can support
Compression	null (0x00)	No compression (per modern TLS)
SNI Extension	example.com	Server Name Indication — reveals the target hostname
Supported Versions	TLS 1.2, TLS 1.3	Versions acceptable to client
Supported Groups	x25519, secp256r1, etc.	Elliptic curves for key exchange

5.6 Server Hello — Observed Details

Field	Observed Value	Explanation
Handshake Type	2 (Server Hello)	Server's response to Client Hello
Selected TLS Version	TLS 1.3 (or 1.2)	Agreed-upon version
Server Random	32 bytes (hex string)	Server's random for key derivation
Session ID	Varies	May match client's session ID
Selected Cipher Suite	TLS_AES_256_GCM_SHA384	Algorithm chosen for the session
Server Certificate	X.509 (base64 DER)	Server identity proof; check CN/SANs
Certificate Validity	Not Before / Not After	Certificate date range
Issuer CN	DigiCert / Let's Encrypt etc.	Certificate Authority that signed it

5.7 What the TLS Packets Reveal

- Even though data is encrypted, the Server Name Indication (SNI) extension in Client Hello reveals the target domain in plaintext (unless ECH — Encrypted Client Hello — is used)
- The selected cipher suite indicates the encryption strength of the session
- Server certificates expose metadata: issuer, validity period, and subject alternative names
- TLS version negotiation can reveal downgrade attacks (e.g., forcing TLS 1.0 usage)
- Anomalies such as self-signed or expired certificates can indicate MITM (Man-in-the-Middle) attacks



Security Insight: TLS protects the payload — making the actual request and response content unreadable to attackers. This is the fundamental protection that HTTPS provides over plain HTTP.

6. Comparative Analysis

The table below summarises the key characteristics of the three protocols analysed in this project:

Attribute	HTTP	DNS	TLS
Default Port	80 (TCP)	53 (UDP/TCP)	443 (TCP)
Transport Protocol	TCP	UDP (mostly)	TCP
Layer	Application (L7)	Application (L7)	Session (L5)
Encryption	None (plaintext)	None by default	Full payload encryption
Visibility in Wireshark	Full content visible	All queries/responses visible	Handshake visible; payload encrypted
Security Risk	High — data exposed	Medium — domains exposed	Low — content protected
Filter Used	http.request.method == "GET"	dns	tls
Key Identifiers	Method, Host, URI, Headers	Query name, Record type, TTL	Cipher suite, SNI, Certificate

7. Findings and Conclusions

7.1 Summary of Findings

- HTTP traffic is transmitted entirely in plaintext, making every request and response readable to anyone on the same network segment
- DNS queries expose browsing habits even when HTTPS is used, unless DoH or DoT is configured
- TLS successfully protects application-layer data, but metadata (SNI, certificate info) is still observable
- Wireshark display filters (http, dns, tls) are effective tools for reducing noise and focusing analysis
- Protocol layers (Frame > Ethernet > IP > TCP > HTTP/TLS) are clearly visible in the Packet Details Pane

7.2 Security Implications

- Always use HTTPS — even small HTTP endpoints can leak sensitive data
- Consider DNS-over-HTTPS (DoH) for browsers to prevent DNS traffic interception
- Monitor for anomalous DNS patterns (NXDOMAIN floods, unusually large responses) as indicators of compromise

- Validate TLS certificates in production environments; self-signed certs can indicate MITM attacks
- Encrypted Client Hello (ECH) is the emerging standard to hide SNI from passive observers

7.3 Conclusion

This project provided practical experience in capturing and interpreting real network traffic using Wireshark. By analysing HTTP, DNS, and TLS packets, a clearer picture emerges of how information flows across the network and where security vulnerabilities may exist. These skills form the foundation of network-level security monitoring and are directly applicable to roles in SOC (Security Operations Center) analysis, penetration testing, and network engineering.

8. References

- Wireshark Foundation. (2024). Wireshark User's Guide. <https://www.wireshark.org/docs/>
- RFC 2616 — Hypertext Transfer Protocol HTTP/1.1. IETF, 1999.
- RFC 1035 — Domain Names: Implementation and Specification. IETF, 1987.
- RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3. IETF, 2018.
- Sanders, C. & Smith, J. (2017). Applied Network Security Monitoring. Syngress.
- OWASP. (2024). Transport Layer Protection Cheat Sheet. <https://cheatsheetseries.owasp.org>

Appendix — Quick Reference: Wireshark Display Filters

Filter Expression	Purpose
<code>http.request.method == "GET"</code>	Show only HTTP GET requests
<code>http.request.method == "POST"</code>	Show only HTTP POST requests
<code>http.response.code == 200</code>	Show only HTTP 200 OK responses
Dns	Show all DNS packets (queries and responses)
<code>dns.qry.type == 1</code>	Show only DNS A record queries
<code>dns.flags.rcode != 0</code>	Show failed DNS responses (errors)
Tls	Show all TLS packets
<code>tls.handshake.type == 1</code>	Show only Client Hello packets
<code>tls.handshake.type == 2</code>	Show only Server Hello packets
<code>ip.addr == 93.184.216.34</code>	Filter traffic to/from a specific IP

tcp.port == 443

Filter traffic on HTTPS port

frame contains "example.com"

Search for domain in any packet

— *End of Report* —